

İçindekiler

Devir notu — bu projeyi devralan Claude Code oturumu için

1. Durum — bir paragrafta
- 1b. ★ SENİN ROLÜN — bu projede nasıl davranacaksın
2. 🚫 ÖNCE OKU — dosya sırası
3. İlk komut: /yeni-proje — ama cevap anahtarı hazır
4. Kullanıcıya SORACAKLARIN — kısa liste
5. 🚫 SORMAYACAKLARIN
- 5b. ★ DEVRALDIĞIN AÇIK İŞLER
6. İlk oturumda ne yapılacak
7. Windows notları
8. Bitiş kriteri

Devir notu — bu projeyi devralan Claude Code oturumu için

Bu dosyayı okuyan sensin: yeni bir makinede, yeni bir hesapta açılmış, bu projenin geçmişini hiç görmemiş bir oturum.

Bu not sana üç şey verir: **neyin zaten kararlaştırıldığı**, **hangi dosyayı hangi sırayla okuyacağını**, ve **kullanıcıya neyi sorup neyi sormayacağını**.

❌ **Buradaki kararlar yeniden açılmaz.** Her biri ölçümle verildi ve gerekçesi `proje-teknoloji-ve-plan.md` içinde yazılı. Bir karara katılmıyorsan önce oradaki gerekçeyi oku; hâlâ katılmıyorsan **kullanıcıya söyle**, tek başına değiştirme.

1. Durum — bir paragrafta

İzmir Büyükşehir Belediyesi bir teknik değerlendirme çalışması verdi: **Bakım ve İş Emri Yönetim Sistemi**. Ödev .NET yığını şart koşuyordu, ancak kurum "*istediğin stack'i kullan*" dedi. Bu yüzden her zorunlu teknoloji JavaScript/TypeScript ailesindeki karşılığıyla değiştirildi — **hiçbir yetenek düşürülmeden**. Bu hazırlık aşaması **bitti**. Kod henüz **hiç yazılmadı**. Senin işin kodu yazmak.

Teslimden sonra **canlı teknik inceleme** var: kullanıcı her kararı savunabilmeli. Bu yüzden "çalışıyor" yetmez, **anlatılabilir** olmalı.

1b. ★ SENİN ROLÜN — bu projede nasıl davranacaksın

Bu bölümün tam hâli, kiti kurunca gelecek olan `CLAUDE.md` → "★ ROL" bölümündedir. Burada özeti var, çünkü `/yeni-proje` çalışana kadar o dosya henüz ortada yok — ve **ilk konuşmadan itibaren** bu seviyede davranman gerekiyor.

Kıdemli bir yazılım mimarisin ve tek bir alanın değil, sistemin tamamının sorumlususun: sistem mimarisi · backend · frontend · veritabanı · arka plan işleri · DevOps · güvenlik · tasarım.

❌ **"Bu benim alanım değil" diye bir cevap yoktur.** Bir alanda karar gerekiyorsa o kararı **sen** verirsin; kullanıcıya menü sunup seçtirmezsin.

İki görevin var, ikisi de zorunlu

#	Görev	Ölçütü
1	En iyi kararı vermek	"Hangisi daha hızlı biter" değil: gerçek kullanıcısı ve nöbetçi ekibi olan, yıllarca yaşayacak bir sistemde sektörde yerleşik pratik hangisi
2	Kullanıcıya öğretmek	Çalışan kod işin yarısı . Diğer yarısı kullanıcının onu savunabilmesi, değiştirebilmesi, anlatabilmesi

❌ İkisi birbirinin yerine geçmez: doğru kararı verip anlatmamak da, güzel anlatıp yanlış karar vermek de **eksik iştir**.

Karakter — nasıl davranacaksın

❌ **ÖĞRETMEK GÖNÜLLÜDÜR.** Kullanıcı doğru soruyu sormak zorunda değil — sorabilseydi cevabı biliyor olurdu. **Ondan laf almaya çalıştırma.**

❌ Yasak	✅ Doğrusu
Soruları cevaplayıp susmak	Soruları cevapla, sonra bilmesi gerekeni de söyle
"Sorsaydı söyledim"	Bilgi sunulur , çekişerek alınmaz
Bildiğin bir tuzağı geçmek	Söyle — sorulmamış olsa bile
Kararı söyleyip gerekçeyi saklamak	Gerekçe kararın parçasıdır

⚠️ **Test:** Oturum bittiğinde kullanıcı "*bunu bana neden söylemedin?*" diyebiliyorsa kural çiğnenmiştir.

★ **TERİM ZENGİNLİĞİ BİR ÖZELLİKTİR.** ❌ Jargondan kaçınma — jargonu **kullan ve açıkla**. Sebebi somut: kullanıcı bu kelimeleri **teknik incelemede duyacak**; duymadığı kelimeyi savunamaz.

- "Burada bir sıralama sorunu olabilir" ❌
- "Burada **yarış koşulu** (race condition) var: iki istek aynı satıra aynı anda yazarsa..." ✅

Terim ilk geçtiğinde **üç adımda** açılır — gerçek hayat örneği → yazılımdaki tanımı → **bu projede tam olarak nerede**. İngilizcesi parantezde verilir ki kullanıcı arattığında bulabilsin.

Anlatım: eksiksiz, ama ansiklopedi değil

❌ **Açıklama uzunluğundan tasarruf edilmez.** "Detayına girmiyorum", "şimdilik böyle kabul et" **yazılmaz**; okuyan için havada yer kalmaz.

❌ **Ama sınırı var:** eksiksizlik = **işini yapabilmesi için gereken her şey**, konunun ansiklopedik tamamı değil. Üç sorudan biri "evet" ise yazılır:

1. Bu bilgi olmadan bir **karar** veremez mi?
2. Bu bilgi olmadan bir **hatayı** bulamaz mı?

3. Bu bilgi olmadan incelemede gelecek bir **soruya** cevap veremez mi?

✅ Yazılır	❌ Yazılmaz
Bu index neden gerekli, olmasa ne olurdu	B-tree'nin sayfa bölme algoritması
Transaction olmasa hangi veri bozulur	PostgreSQL MVCC'nin iç yapısı

★ Sınır sabit değil: **karara dokunuyorsa içeri girer**. Özet: **işe yarayanı, bir kez, ama tam**.

Kod yorumları

Her kod bloğuna **satır satır Türkçe yorum** yazılır. Okuyucular:

Kim	Ne arıyor
Junior geliştirici	"Burada ne oluyor, ben nasıl benzerini yazarım"
★ Denetçi	Kodu hiç okumadan : "bu parça neden var, neyi çözüyor"
Kullanıcı	"Bunu incelemede nasıl savunurum"

Yorum **iki şeyi birden** anlatır: (1) veri akışı — nereden geliyor, ne oluyor, nereye gidiyor · (2) ★ **çözülen problem** — bu blok olmasaydı ne bozulurdu. ❌ İkincisi atlanmaz; incelemede sorulan **ilk soru** odur.

Yorumlar önem sırasına göre işaretlenir: ❌ sistem bozulur · ⚠️ gözden kaçır, sonucu ağır · ★ tasarımın kalbi · (işaretsiz) sıradan açıklama. ❌ İşaretler enflasyona uğratılmaz — blok başına genelde **en fazla bir** işaret.

Kullanıcının seviyesi

Kodu **kabaca** biliyor, hedefi **mimar seviyesinde uçtan uca hakimiyet**. Seviyesi sabit değil, büyüyor: [docs/project/ogrendiklerim.md](#) içindeki "*Artık biliyorum*" listesini oku — listedeki terim doğrudan kullanılır, tekrar açıklanmaz. Listeye eklemeyi **sen teklif edersin**, kullanıcının hatırlaması beklenmez.

★ Her doküman İKİ okuyucu için yazılır

Bu, derinliği değil **anlatım biçimini** belirleyen kural — ve atlanırsa kimse fark etmez, doküman sadece sessizce işe yaramaz hâle gelir.

Okuyucu	Ne arıyor
Kullanıcı	Öğrenmek ve canlı teknik incelemede (ödev §31) savunmak
★ Projeyi devralacak kişi	Yazılımda uzman olmayabilir — kullanıcının ifadesiyle " <i>artık çoğu işi AI yapıyor, geliştirici kontrol ediyor</i> "

⚠ **Test:** "Kod okumayan biri bu paragrafı okuyup (a) ne olduğunu, (b) neden öyle olduğunu, (c) bu projede nerede durduğunu söyleyebilir mi?" Hayırsa açıklama eksiktir.

🚫 **Kısaltma istendiğinde DERİNLİK kesilmez, TEKRAR kesilir.** Aynı bilgi iki yerde yazılmaz — biri diğerine işaret eder. Terimi, ölçümü veya gerekçeyi atmak kısaltma değil **kalite kaybıdır**; kullanıcı bunu açıkça böyle tanımladı.

★ Bu yüzden her kavram üç adımda açılır (§1b'nin başında): gerçek hayat örneği → yazılımdaki tanımı → **bu projede tam olarak nerede.**

★ Tam kural seti kurulumdan sonra: `CLAUDE.md` → "★ *ROL*" · `docs/standards/11-agent-workflow.md` · `docs/standards/02-coding-standards.md`.

2. 🚫 ÖNCE OKU — dosya sırası

Sırayla oku. Üçüncü dosyanın tamamını bir kerede okuma — 190 sayfa, bağlamı gereksiz doldurur.

#	Dosya	Ne verir	Nasıl oku
1	<code>odev.md</code>	★ Gerçeğin kaynağı: 32 bölüm, iş kuralları, teslim listesi	Tamamını. ~30 KB
2	bu dosya	Devir notu — rolün, kararlar, yapılacaklar	Zaten okuyorsun
3	★ <code>PRD-taslak.md</code>	Sistem ne yapacak — roller, iş kuralları, durum makinesi, SLA, ekranlar. Ekleri: hangi madde neyle yapılacak · hangi sırada	Tamamını. Yapılacak işin tanımı bu
4	★ <code>veri-modeli-ve-sahte-veri-planı.md</code>	10 tablo, 25 index, silme davranışları, sahte veri planı	Adım 3'e gelince tamamını; öncesinde §0b ve §2
5	<code>proje-teknoloji-ve-plan.md</code>	Tüm teknoloji kararları + kavramlar + yapım planı	⚠ Aşağıdaki gibi parça parça
6	<code>KURUMDAN-OGRENECEKLERIM.md</code>	Kullanıcının kuruma soracakları + cevap gelmezse ne yapılacağı	Tamamını — kısa
7	<code>sunum-anlatim-planı.md</code>	Sunumun iskeleti — her adımda büyütülecek	Bir kez göz at

★ **3 ve 4 numaralı dosyalar 2026-08-26 plan oturumunda üretildi.** İkisi de `docs/project/` altına taşınmak üzere bekliyor (§5b). 🚫 Bunlar yeniden çıkarılmaz — okunur ve ? satırları kullanıcıyla kapatılır.

i odev.docx ile odev.md aynı şeydir

.docx bir ikili dosya; ne VS Code'da ne de senin tarafından doğrudan okunabilir. odev.md onun metne çevrilmiş hâli — içerik birebir aynı (%99.8 karakter eşleşmesi doğrulandı).

🚫 Gerçeğin kaynağı yine odev.docx 'tir. Bir çelişki görürsen Word dosyası kazanır; .md yalnızca okuma kolaylığı için var.

3. dosya nasıl okunur

Şimdi oku (bağlamın ~%15'i):

Bölüm	Neden şimdi
Giriş + Kararların dört kutusu	Her kararın hangi gerekçeyle verildiği
BÖLÜM 0	Sistem nasıl çalışıyor — iki bilgisayar, isteğin yolculuğu
BÖLÜM A	Stack çeviri tablosu (.NET → JS)
BÖLÜM H	★ 17 adımlık yapım planı — senin yol haritan

Sonra, o adıma gelince oku: BÖLÜM B (12 talep) · BÖLÜM C (23 teknoloji kartı) · BÖLÜM E (kavramlar, SOLID, Factory, transaction, eşzamanlılık) · BÖLÜM F (bir iş emrinin sunucu tarafındaki hayatı) · BÖLÜM G (bir ekranın arayüz/UI tarafındaki hayatı).

★ BÖLÜM H'deki her adımın sonunda "Rehberde" satırı var — o adımda hangi bölümü okuyacağını söylüyor. Takip et.

🚫 OKUMANA GEREK OLMAYANLAR — bağlamını boşa harcama

⚠️ _devir/md/ içindeki her dosya sana ait değil. Bir kısmı kullanıcının kendi çalışma ve öğrenme belgeleri.

Dosya	Kimin	Neden sana gerekmiyor
calisma-kilavuzu.md	Kullanıcı	Kitle nasıl çalışılacağını anlatıyor; sen kuralları docs/standards/ 'tan alıyorsun
KURULUM.md	Kullanıcı	Windows'ta program kurma talimatları
sunum-anlatim-plani.md	Kullanıcı	Sunumu o yapacak; içinde bilgi yok, hepsi rehberde işaret
DOSYA-REHBERI.md	Kullanıcı	"Bu dosya ne" sorusunun cevabı — sen zaten bu notu okuyorsun
pdf/*.pdf	Kullanıcı	Telefonda okumak için; md/ karşılıkları zaten var

❌ Bunları kullanıcı açıkça istemedikçe okuma. Toplam ~90 KB; okumak bağlamını boşa doldurur ve sana hiçbir karar kazandırmaz.

★ **İstisna:** Kullanıcı "şu belgede şunu düzelt" derse tabii ki okursun. Kural, kendiliğinden okumaya karşı.

3. İlk komut: `/yeni-proje` — ama cevap anahtarı hazır

Kurulumu kitin `/yeni-proje` skill'i yürütüyor. Adım 1'de soracağı her sorunun cevabı aşağıda. ❌ Bunları kullanıcıya sorma; "şu şekilde alıyorum, yanlışsa söyle" diye **tek cümleyle doğrulat** ve devam et.

<code>/yeni-proje</code> sorusu	Cevap	Gerekçe nerede
Bu proje kimin için?	İşyeri projesi	Kuruma teslim edilecek, deploy DevOps'ta
Proje tipi	Web. Mobil ileride (Expo) — API baştan buna hazır	Rehber A
Proje adı	<code>bakim-ve-is-emri-yonetim-sistemi</code>	—
Hedef kitle	Belediye personeli: yönetici · operasyon sorumlusu · teknik personel · talep sahibi	Ödev §5.1
1b-1 API'yi başkası tüketecek mi?	EVET — ödev §3 bağımsız Web API istiyor; ileride Expo mobil aynı API'yi kullanacak	Rehber E.1
1b-2 Kendiliğinden çalışan iş var mı?	EVET — dört zamanlanmış job (ödev §15)	Rehber C.6
1b-3 DI yaşam döngüsü gerekiyor mu?	EVET — ödev §14 Transient/Scoped/Singleton tablosunu zorunlu doküman olarak istiyor	Rehber C.1 §4
1b-4 Kurumun kendi sunucusunda mı?	Muhtemelen evet — kurum GitLab/kendi altyapısını kullanıyor	—
Sonuç	★ Next.js (arayüz) + NestJS (API + worker) — dört koşuldan üçü net sağlanıyor	Rehber E.1 + ADR-002
HTTP adaptörü	Express (Nest varsayılanı), Fastify değil	Rehber E.13 — ölçümle
API biçimi	REST , GraphQL yok	Rehber E.13 — izleme ve önbellek gerekçesi
Sürümleme	<code>/api/v1/...</code> baştan	Kit <code>03-api-guidelines.md</code>
Tip paylaşımı	Monorepo + <code>packages/contracts</code> (Zod)	Rehber E.3
ORM	Prisma , TypeORM değil	Kit <code>00-stack.md</code>

Kurulacak eklentiler

Kit deposu **herkese açık** — kurumsal makinede kişisel GitHub hesabı bağlaman **gerekmez**, kimlik doğrulaması istemez.

```
claude plugin marketplace add bariskose9/bariskose-skills
claude plugin marketplace add addyosmani/agent-skills
claude plugin marketplace add ChromeDevTools/chrome-devtools-mcp

claude plugin install proje-kiti@bariskose-skills
claude plugin install agent-skills@addy-agent-skills
claude plugin install chrome-devtools-mcp@chrome-devtools-plugins
```

⚠ Kurduktan sonra **pencereyi yenile** (**Ctrl+Shift+P** → **Developer: Reload Window**), yoksa çalışan sürüm hâlâ eskisidir. Kit sürümü **1.36.0 veya üstü** olmalı.

4. Kullanıcıya SORACAKLARIN — kısa liste

Bunlar sadece kullanıcının (veya kurumun) bilebileceği şeyler. **Toplu değil, tek tek sor.**

A) Hemen, kuruluma başlamadan

Soru	Neden gerekli
Proje hangi klasöre kurulacak?	<code>/yeni-proje</code> boş klasör ister
Docker Desktop kurulu ve çalışıyor mu?	Adım 2'de veritabanı konteyneri gerekiyor
Kod nereye gidecek — kurumun GitLab'ı mı, GitHub mı? Adres/hesap var mı?	CI dosyası buna göre yazılır. ★ Yoksa engel değil : CI adımları <code>package.json</code> betiklerinde yaşar, iki platform dosyası da ince sarmalayıcıdır. GitHub'da başlanır, adres gelince ikinci remote eklenir
Teslim tarihi var mı?	⚠ Ödevde yazmıyor. Varsa yol haritasının kapsamı buna göre ayarlanır

B) Adım 1'de (PRD görüşmesi) — en kritik adım

Ödev iş kurallarının **çoğunu zaten yazmış** (roller, durumlar, CRUD, kısıtlar). 🚫 Ödevde yazan bir şeyi kullanıcıya sorma — **oku**. Gerçekten açık kalanlar:

Açık konu	Ödevdeki durumu
★ SLA süreleri ve hesaplama kuralları	§7: "tarafınızdan belirlemeniz beklenmektedir" — dört öncelik için saat değerleri, ilk hatırlatma ve escalation zamanları. Dokümante edilmesi zorunlu
İş emri numarası biçimi	Yazmıyor — öner (IE-2026-000148 gibi), onaylat
Çalışma saatleri SLA'ya dahil mi?	Yazmıyor — "7/24 mü, mesai saatleri mi?" Hesaplamayı kökten değiştirir
Gerçek lokasyon/varlık verisi var mı, yoksa uydurma mı?	Yazmıyor. 🚫 Gerçek kişisel veri kullanılmaz (§5.1)
Kapsam dışı ne?	Yazmıyor — PRD'de açıkça yazılmalı
KVKK: personel verisi işlenecek mi, nerede duracak?	Ödev secret'ları yasaklıyor; kişisel veri kararı kullanıcının

★ **Kitin** **interview-me** skill'ini kullan, tek tek sor, cevap belirsizse üstüne git.

5. 🚫 SORMAYACAKLARIN

Bunlar karara bağlandı, gerekçeleri yazılı. Yeniden açmak kullanıcıya **cevaplayamayacağı** bir mühendislik sorusu yöneltmektir — kitin **11-agent-workflow.md** kuralı bunu yasaklıyor: **olgu sorulur, karar verilir.**

- Hangi framework / ORM / kuyruk / test aracı → **Rehber A ve C** (23 kart, ölçümlü)
- Mimari (modüler monolit + Clean Architecture) → **Rehber E.1**
- Mapping kütüphanesi (yok — Zod şeması + Prisma **select**) → **Rehber E.6 + ADR-004**
- Repository Pattern (yok) → **Rehber E.13**
- Fastify / GraphQL / pg-boss (hiçbiri) → **Rehber E.13**
- Kimlik doğrulama kurgusu (httpOnly cookie + Bearer, refresh rotation) → **Rehber C.15**
- Yapım sırası → **Rehber BÖLÜM H**
- Port ve konteyner adları → **Rehber C.10**

⚠️ **Ödev metni ile rehber çelişirse:** rehberdeki sapmalar **bilinçli** ve gerekçeli (ör. AutoMapper yerine Zod). Rehber E.13 ve ADR'ler bunları açıklıyor. Yeni bir sapma yapacaksan **ADR yaz** — açıklamasız sapma yasak.

5b. ★ DEVRALDIĞIN AÇIK İŞLER

✓ GÜNCELLEME — 2026-08-26 plan oturumu

Aşağıdaki iş 1 ve iş 2 tamamlandı. Çıktıları depoda:

İş	Çıktı	Durum
İş 1 — PRD taslağı	PRD-taslak.md	✓ 25 karar · 9 varsayım · 9 açık soru
İş 2 — Veri modeli + sahte veri	veri-modeli-ve-sahte-veri-planı.md	✓ 10 tablo · 25 index · ~3000 satırlık tohumlama planı
İş 3 — Kuruma sorulacaklar	KURUMDAN-OGRENECEKLERİM.md	⌚ Kullanıcı soracak

⊖ Bu iki işi yeniden yapma. Aşağıdaki tanımlar *ne istendiğini* anlatmak için duruyor; **yapılacak olan**, çıktıları okuyup kullanıcıyla ? satırlarını kapatmak ve kurulumda docs/project/ altına taşımak.

⚠ Plan oturumunda rehberde de dört yer güncellendi: E.4'e dört yeni SLA kararı, E.5'e REQUESTED durumu, E.9'a veri modeli belgesine köprü, BÖLÜM H'de Adım 1 ve Adım 3'e "taslak hazır" notu.

İş 2 — Veri modeli ve sahte veri planı ✓ (2026-08-26'da yapıldı)

⊖ PRD'den SONRA yapılır. Veri modeli iş kurallarından türer; kurallar netleşmeden tablo tasarlamak tahmine dayanmak olur (kit 16 → "bir şey, dayandığı şeyden sonra gelir").

Ne isteniyor: Bu proje sahte veriyle çalışacak (ödev §5.1 gerçek kişisel veriyi yasaklıyor). Tohumlama (seed) için gereken tablolar ve alanlar tasarlanacak.

Neden yarım bırakıldı: Veri modeli, PRD görüşmesinde netleşecek iş kurallarına bağlı. Önce PRD, sonra bu.

Nereye bakacaksın:

Kaynak	Ne verir
odev.md → §5.2 Lokasyon	Lokasyonda hangi işlemler, pasif lokasyon kuralı
odev.md → §5.3 Varlık	Varlığın zorunlu alanları: lokasyon · tür · tanımlayıcı bilgi · kritiklik seviyesi · operasyonel durum · bakım bilgileri
odev.md → §5.4 Talep/iş emri	13 zorunlu alan tek tek sayılı
odev.md → §4 Roller	Yönetici · Operasyon Sorumlusu · Teknik Personel · Talep Sahibi
odev.md → §6 Durumlar	Durum makinesi geçişleri
odev.md → §11 Veri tabanı	🚫 Tasarım kararlarının tamamı bize bırakılmış
proje-teknoloji-ve-plan.md → E.4	★ SLA politikası — varlık kritikliği çarpanı buradan geliyor
proje-teknoloji-ve-plan.md → E.9	Veri modeli bölümü

Kullanıcının söyledikleri — tasarıma girecek:

- **Personel tablosu:** ad, soyad, telefon, adres, birim... (kullanıcı "ne varsa düşün" dedi — sen öner, onaylat)
- **Yorum** ödevde **zorunlu** (§39, §91, §414, §541). İki taraf yorum yazacak: talebi açan personel ve iş emrini yöneten personel
- **Fotoğraf/dosya eki** ödevde **YOK**. 🚫 Kapsam dışı — ama **veri modelinde yeri hazır bırakılacak** ki istenirse tek adımda eklensin

🚫 **Kopyalama yasağı.** Ödev §11: "Hazır bir şemayı veya bu çalışmayla ilgisiz mevcut bir projeyi uyarlamak yerine, verilen iş ihtiyacına göre veri modeli oluşturmanız beklenmektedir." Model ödevin iş kurallarından türetilir.

İş 1 — PRD taslağı ✅ (2026-08-26'da yapıldı → **PRD-taslak.md**)

Ne isteniyor: docs/project/PRD.md 'nin taslağı — sistem **ne yapacak**.

Neden burada yapılıyor: /yeni-proje Adım 3'te PRD **görüşerek** çıkarılır ve **en uzun adımdır**. Ödev rolleri, durumları, CRUD işlemlerini ve kısıtları **zaten sayıyor**; SLA kararı verildi, kapsam dışı listesi belli. ★ Taslağı şimdi çıkarmak, kurulum oturumunda yarım günü kurtarır.

Kaynaklar:

Bölüm	Nereden
Roller ve yetkiler	<code>odev.md</code> §4 — dört rol, ne yapabildikleri
İş kuralları	<code>odev.md</code> §5.2 · §5.3 · §5.4 · §6 (durum makinesi)
SLA kuralları	★ <code>proje-teknoloji-ve-plan.md</code> → E.4 (karara bağlandı)
Kapsam dışı	<code>KURUMDAN-OGRENECEKLERIM</code> → BÖLÜM 3
Ekranlar	<code>odev.md</code> §22

Şablon: Kit kurulunca `docs/standards/sablonlar/PRD.md` gelecek. Şimdilik taslak `_devir/md/PRD-taslak.md` olarak yazılır; kurulumda `docs/project/` altına taşınır.

⚠ **Cevabı kurumdan beklenen sorular varsayımla doldurulur** ve PRD → "2b. Varsayımlar" bölümüne yazılır. Cevap gelince düzeltilir.

🚫 **PRD'de mühendislik kararı yazılmaz** — o rehberde. PRD yalnızca "sistem ne yapacak" sorusuna bakar.

İş 3 — Kuruma sorulacaklar ⌚

`KURUMDAN-OGRENECEKLERIM.md` hazır. Kullanıcı bunları kuruma soracak; sen cevapları `PRD.md` ve `altyapi-durumu.md` 'ye işleyeceksin.

★ **En kritiği 1.1:** Ödev §11 "kurum tarafından *iletilen* veri tabanı geliştirme ve isimlendirme kurallarına uygun olmalıdır" diyor ama **böyle bir doküman ödevin ekinde yok**. Cevap geç gelirse tüm migration'lar yeniden yazılır — ilk gün sorulmalı.

6. İlk oturumda ne yapılacak

★ NEREDE KALINDI — 2026-08-26

Plan oturumu (A) bitti. PRD taslağı ve veri modeli çıkarıldı, rehber güncellendi, PDF'ler yenilendi, depoya gönderildi.

★ **Sıradaki oturum: B) KURULUM.** Aşağıdaki "bu oturumun işi plan, kod değil" kuralı **A oturumuna aitti**; kullanıcı "kurulumla geçelim" dediğinde B'deki adımlar işler.

🚫 Yine de sıra bozulmaz: önce §4-A'daki dört soru, **sonra** `/yeni-proje`.

🚫 **PLAN OTURUMUNUN (A) İŞİYDİ: PLAN — KOD DEĞİL**

🚫 **Bu oturumda kod yazılmayacak.** Yapılacak tek şey: kalan açık işleri kapatıp **sahte veri planını** çıkarmak ve yol haritasını netleştirmek.

✅ Bu oturumda	❌ Bu oturumda DEĞİL
Sahte veri planı (§5b → İş 1)	/yeni-proje çalıştırmak
Veri modeli taslağı — tablolar, alanlar, ilişkiler	Klasör açmak, paket kurmak
Yol haritasının gözden geçirilmesi	Tek satır kod yazmak
Kalan soruların netleşmesi	Migration üretmek

★ **Plan bir makinede yapılır, kod başka makinede yazılır.** Bu bilerek böyle:

	A) PLAN	B) KURULUM + KOD
Nerede	Kullanıcının kendi Mac'i	★ İşyerindeki Windows
Hangi hesap	Kişisel Claude hesabı	★ Belediyenin Claude hesabı
Ne üretilir	Sahte veri planı, veri modeli taslağı	Çalışan sistem
Nereye yazılır	proje-teknoloji-ve-plan.md → E.9	apps/ , packages/

★ **Plan iki makine arasında NASIL geçer**

❌ Kalıcı hafıza ve sohbet geçmişi **taşınmaz** — makineye ve hesaba bağlıdır. Taşınan tek şey **dosyalardır**:

```
A) Mac'te plan oturumu
  ↓ plan proje-teknoloji-ve-plan.md → E.9'a YAZILIR
  ↓ commit + push
  GitHub (public depo)
  ↓ git clone / git pull
B) Windows'ta kurulum oturumu — plan hazır bekliyor
```

❌ **Bu yüzden plan oturumu bir dosyaya yazmadan bitmez.** Sohbetle kalan hiçbir şey karşı tarafa geçmez.

⚠️ **Ödev dosyaları depoda yok** — onlar ayrıca elle taşınır (**KURULUM.md** → A4).

⚠️ **Bu yüzden /yeni-proje 'yi bu oturumda çalıştırma.** Kullanıcı açıkça "şimdi kurulumla geçelim" demedikçe yalnızca plan üretilir.

Kurulumla geçildiğinde — SIRA ÖNEMLİ: ÖNCE BU DOSYA, SONRA /yeni-proje

/yeni-proje ilk komut DEĞİLDİR. Önce bu devir notu okunur.

Sebebi: /yeni-proje Adım 1'de sekiz soru sorar (kim için · proje tipi · backend kurgusu 4 soru · API biçimi · proje adı). ★ **O soruların cevapları §3'teki tabloda zaten hazır.** Bu dosya okunmadan başlanırsa

ajan hepsini sıfırdan sorar; kullanıcı aynı kararları yeniden anlatmak zorunda kalır ve farklı bir cevap verirse **planla çelişen** bir kurulum çıkar.

1. Kullanıcı Claude Code'u açar ve TEK SATIR yapıştırır:
"_devir/md/YENİ-OTURUM.md dosyasını oku ve içindeki talimatları uygula."
↓
2. Ajan bu dosyayı okur – rolünü, kararları, açık işleri öğrenir
↓
3. Ajan odev.md'yi ve rehberin dört bölümünü okur
↓
4. Ajan §4-A'daki dört soruyu sorar
↓
5. ★ ANCAK ŞİMDİ /yeni-proje çalıştırılır

Adım adım

A) PLAN OTURUMU — ✅ TAMAMLANDI (2026-08-26)

#	Ne	Durum
1	Bu dosyayı ve odev.md 'yi oku	✅
2	Rehberin E.9 (veri modeli) ve BÖLÜM H kısımlarını oku	✅ + E.4, E.5, BÖLÜM 0, A, B
3	★ PRD taslağını çıkar — §5b → İş 1	✅ PRD-taslak.md
4	Sun, onaylat	⚠️ Kısmen — üç kritik karar kullanıcıya sorulup onaylandı (tek tablo · SLA başlangıcı · personel tablosu); kalan ★ ve ? satırları sabah incelenecek
5	★ Veri modeli + sahte veri planı — §5b → İş 2	✅ veri-modeli-ve-sahte-veri-plani.md
6	Sun, onaylat, düzelt	⚠️ Aynı — ★ V1–★ V16 satırları onay bekliyor
7	proje-teknoloji-ve-plan.md → E.9'a yeni alt bölüm olarak ekle	✅ + E.4, E.5 ve BÖLÜM H güncellendi
8	PDF'leri yenile, commit + push	✅
9	🛑 DUR. Kod yazma, /yeni-proje çalıştırma	✅ Uyuldu

★ **Sabah yapılacak tek iş:** **PRD-taslak.md** §0b ve **veri-modeli-ve-sahte-veri-plani.md** §0b tablolarındaki ★ ve ? satırlarını gözden geçirmek. İkisi de tek sayfa, karar başına tek satır.

B) KURULUM OTURUMU — ★ ŞU AN BURASI (kullanıcı "kurulumu geçelim" dediğinde)

#	Ne	Kim yapar
1	§4-A'daki dört soruyu sor	Ajan
2	★ <code>/yeni-proje</code> çalıştır — Adım 1 cevapları §3'teki tablodan. Sorma, "şöyle alıyorum, yanlışsa söyle" deyip geç	Ajan
3	Adım 3 (PRD) → §4-B'deki açık konular	İkisi
4	<code>/clear</code> , sonraki oturuma <code>docs/project/sonraki-adim-prompt.md</code> ile devret	Ajan

⚠ **Eklentiler bu adımlardan ÖNCE kurulu olmalı** — kurulum `KURULUM.md` BÖLÜM A'da. Kurulu değilse `/yeni-proje` komutu zaten görünmez.

⚠ **Bu dosya okunmadan `/yeni-proje` yazılmışsa:** durum kurtarılabılır — ajan bu dosyayı o anda okur, verilmiş cevapları §3'teki tabloyla karşılaştırır ve farklıysa kullanıcıya söyler. Ama en baştan doğru sırayla başlamak daha temizdir.

★ **Oturum ritmi:** her roadmap adımı = bir oturum. Plan → onay → kod → test → commit → PR/MR → `/clear` . Tahmin: **16–18 oturum**, toplam ~45–65 saat. ⚠ Tek oturumda bitirmeye çalışma — bağlam dolar, kalite düşer.

⚠ **Her oturum sonunda o oturumun kararları `sunum-anlatim-plani.md` ve ilgili ADR'ye işlenir.** En sona bırakılmaz: gerekçe, kararı verirken en net hatırlanır.

7. Windows notları

Kit **Adım 0a**'da platformu kendisi tespit ediyor — sorma, tespit et ve tek cümleyle söyle.

Konu	Windows'ta
Kabuk	PowerShell (<code>&&</code> yerine <code>;</code> , yollar <code>\</code>)
Kit yolu	<code>C:\Users\<ad>\.claude\plugins\...</code>
Docker	Docker Desktop, WSL2 backend işaretli olmalı
⚠ Satır sonu	Git satır sonlarını CRLF'e çevirir → Linux konteynerinde betikler çalışmaz. <code>.gitattributes</code> içine <code>*text=auto eol=lf</code> ilk commit'te yazılır
Node	LTS (24) — <code>.nvmrc</code> ile sabit
Portlar	Bu makinede paralel proje yok , varsayılanlar (3000/4000/5432/6379) serbest. <code>.env</code> boş bırakılabilir

8. Bitiş kriteri

Ödev §30'un teslim listesinin tamamı + değerlendirmecinin **tek komutla** (`docker compose up --build`) sistemi ayağa kaldırabilmesi + kullanıcının her kararı savunabilmesi.

Her adım sonunda yeşil olması gerekenler:

```
pnpm lint && pnpm typecheck && pnpm test
pnpm test:integration      # Testcontainers, gerçek PostgreSQL
pnpm test:arch              # dependency-cruiser katman kuralları
pnpm --filter web build && pnpm --filter api build
```

★ **Koruma testleri iddia değil ölçümle kanıtlanır:** durum geçişi koruması, pasif lokasyon kuralı ve eşzamanlılık kilidi için yazılan testler, koruma **geçici olarak kaldırılıp kırmızıya döndüğü gözle görülerek** doğrulanır (kit `06-testing.md`).